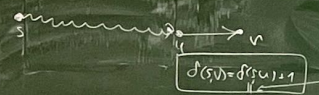
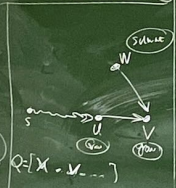


$$d(v) = d(s, v)$$

Der Beweis der Korrektheit: zu zeigen: $d(v) = \delta(s, v)$ bei der Terminierung
 Im Beweis: sei $v \in V$ der Knoten mit $d(v) > \delta(s, v)$ bei der Terminierung
 und dem kleinsten Abstand (Widerspruchsargument).



Wir betrachten den Moment der Ausführung in dem die Kante (u, v) besucht wird. In diesem Moment ist $d(u) = \delta(s, u)$ (d.h. bereits gesetzt)
 $d(v) \leftarrow$ nachdem diese Kante gesetzt ist, hat man diese Gleichung.



In diesem Moment ist Farbe[v] Weiss, Grau oder Schwarz.

Ist Farbe[v] = Weiss: wie der Algorithmus setzt $d[v] = d[u] + 1$ unter:
 $d(u) = \delta(s, u)$ und $\delta(s, v) = \delta(s, u) + 1$ gelten. Es folgt: $d(v) = d(u) + 1 = \delta(s, u) + 1 = \delta(s, v)$
 Widerspruch.

Ist Farbe[v] = Grau: Dann ist $d(v)$ bereits gesetzt und v kann in Q oder U sodass $d(v) \leq d(u)$ gilt.
 $d(v) \leq d(u) + 1 = \delta(s, u) + 1 = \delta(s, v)$, Widerspruch zur Annahme $d(v) > \delta(s, v)$.
 Ist Farbe[v] = Schwarz, so ist v bereits in der Warteschlange und v wurde von einem Knoten w aus entdeckt, der bereits die Schlange verlassen hat und Schwarz ist.
 Dann gilt: $d(v) = d(w) + 1 \leq d(w) + 1 = \delta(s, w) + 1 = \delta(s, v)$, Widerspruch zu $d(v) > \delta(s, v)$.

Ist Farbe[v] = Schwarz, dann war v bereits in Q und hat Q verlassen, sodass $d(v) \leq d(u) = \delta(s, u) = \delta(s, v) - 1$, Widerspruch zu $d(v) > \delta(s, v)$. □

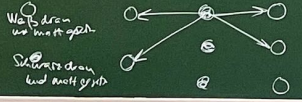
Wenn die DFS(s) geklappt ist, wie bestimmt man für ein $v \in V$ mit $d(v) < \infty$ den entsprechenden kürzesten (s, v) -Pfad?

Pfad(v):
 if $\pi(v) = \text{NIL}$:
 return s
 else:
 Pfad($\pi(v)$) + [v]

das kann man auch iterativ umsetzen

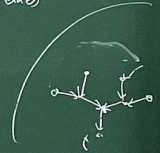
Die Menge $A_{\pi} := \{(v, w) : v \in V \text{ mit } \pi(v) \neq \text{NIL} \text{ und } \pi(v) = w\}$ und $V_{\pi} := \{v \in V : \pi(v) \neq \text{NIL}\} \cup \{s\}$ sind Knoten- bzw. Kantenmenge eines gerichteten Baums mit der Wurzel s. Das ist der Baum der kürzesten pfade.

- Testinstanzen: DIMACS Shortest path (USA, Teile von USA)
- Anwendung: Berechnung der Spiele



$$P_{\pi} = (V_{\pi}, A_{\pi})$$

Optimale Strategien zu Spielen



4. Tiefensuche.

Das Prinzip wie bei der Breiten Suche (Knoten sind Weiss, Grau und Schwarz). Der Unterschied: die BFS ist "konsequent" und die DFS ist "hängig".
 Container = Stack

```
Tiefensuche(u):
  Farbe[u] = Grau
  For v ∈ N[u]:
    if Farbe[v] = Weiss:
      P[v] = u
      Tiefensuche(v)
  end
  Farbe[u] = Schwarz
```

```
Tiefensuche-initialisieren():
  For v ∈ V:
    Farbe[v] = Weiss
    P[v] = NIL
  end
```

Vollständige-Tiefensuche()

```
Tiefensuche-initialisieren()
For u ∈ V:
  if Farbe[u] = Weiss:
    Tiefensuche(u)
  end
end
```

TS(1) ✓
 TS(2) ✓
 TS(4) ✓
 TS(4) x
 ...



(3)